
The Conversation Feature as Fourteen Algorithms

Pseudocode for the complete flow — engine launch, voice capture, turn generation, chunked playback, analysis and spaced-repetition scheduling

Written for: a reader who needs the mechanism, not the source

Notation: structured pseudocode. Nothing here assumes a language.

System: FluencyOS · Electron + React client, FastAPI + SQLite server

Models: faster-whisper tiny.en · llama.cpp / OpenRouter / Gemini · Pocket TTS

1. Notation

This document states the Conversation feature as fourteen algorithms. They are written to be read on their own — you should not need the source open — but each names the procedure it corresponds to, so any of them can be checked against the code.

Conventions

Form	Meaning
<code>x <- y</code>	Assignment.
<code>CALL P(...)</code>	Invokes another algorithm in this document.
<code>RAISE E</code>	Aborts and propagates to the caller. Nothing after it runs.
<code>ASSERT-DOWNLOADED</code>	Model file present / API key configured, else RAISE.
<code>ASSERT-LOADED</code>	Weights resident in memory, else RAISE. A different check.
<code>NIL</code>	Absence. Used strictly for " <i>not measurable</i> ", never for zero.
<code>EVERY FRAME DO</code>	A loop driven by a timer, not by data. Client-side only.
<code>SIGNAL f()</code>	Fires a callback; the caller does not wait.
<code>AWAIT x</code>	Suspends until an outstanding request completes.
<code>// text</code>	Commentary, not a step.

One convention deserves its own note. `NIL` and `0` are not interchangeable anywhere in this feature. `NIL` means *this could not be measured*; `0` means *this was measured and it was zero*. A text conversation has no speaking rate at all — reporting 0 words per minute would be a false statement about the learner. Several branches that otherwise look redundant exist only to preserve this distinction.

Where each algorithm runs

Tier	Algorithms
Client (React, in the Electron window)	6 Capture-Utterance, 7 Play-Reply
Server (FastAPI, local Python process)	1–5, 8–13
Either (orchestration)	0 Conversation-Lifecycle

The two tiers are separate operating-system processes and communicate only over loopback HTTP with a per-launch token. The client holds no models, no database and no files; every capability it appears to have is a request.

2. Data structures

Five structures carry all the state. Fields marked `?` may be `NIL`.

Session

`id`, `learner`, `scenario` in {free, coffee, job, debate}, `channel` in {voice, text}
`targetWordIds` : list of word id, serialised
`modelId` : what most recently ANSWERED - a record, not a rule
`startedAt`, `endedAt?`, `reportJson?`

Turn

`id`, `session`, `index`, `speaker` in {user, ai}, `text`, `createdAt`
`sttConfidence?` : recogniser certainty - the pronunciation proxy
`speechSeconds?` : REAL duration of recognised speech - not wall clock
`audioPath` : always NIL. Audio is made per chunk, on demand.

TargetWord

`id`, `word`, `definition?`

LlmTarget

one of:

{ `provider`: LOCAL, `option`: model file on disk }
{ `provider`: OPENROUTER | GEMINI, `model`: string, `apiKey?`: string }

Report

`version`, `sessionId`, `turnCount`
JUDGED `grammaticalPrecision`, `errors[<=3]`, `selfCorrections`, `summary`
DETERMINISTIC `wordsPerMinute?`, `fillerRate`, `avgResponseDelay?`, `longestRun`,
`typeTokenRatio`, `aboveLevelWords`, `pronunciationScore?`
RECONCILED `contextualAccuracy?`, `routing[]` of (word, outcome, evidenceTurn?)

Two fields encode decisions worth pausing on.

Turn.audioPath is always NIL. Synthesising a whole reply before returning it meant the learner waited about 30 seconds to see any reply at all, because synthesis runs slower than real time here. Audio became per-chunk and on demand, and this field is the vestige of the older design that Algorithm 8 replaced.

Session.modelId records, it does not govern. Sessions were once pinned to their starting engine. When that engine became unusable — quota spent, key revoked, model deleted — the conversation could never continue and no Settings change could rescue it.

3. Lifecycle and setup

Algorithm 0 is the whole feature in twenty-one lines. Everything after it is one of the procedures it calls.

Algorithm 0 CONVERSATION-LIFECYCLE

the feature end to end

```
ALGORITHM Conversation-Lifecycle(learner)
INPUT   learner, a user with a vocabulary list and a CEFR level
OUTPUT  a report, and an updated review schedule for each target word

BEGIN
1  CALL Launch-Engines(learner)           // once, explicit, user-initiated
2  session ← CALL Start-Session(learner, scenario, channel)
3
4  WHILE learner has not left DO
5      IF channel = VOICE THEN
6          utterance ← CALL Capture-Utterance() // client, hands-free
7          (userTurn, aiTurn) ← CALL Submit-Turn(session, audio := utterance)
8      ELSE
9          text ← text typed by learner
10         (userTurn, aiTurn) ← CALL Submit-Turn(session, text := text)
11     END IF
12
13     display aiTurn.text immediately
14     IF channel = VOICE THEN
15         CALL Play-Reply(aiTurn)           // overlaps with step 4 of next loop
16     END IF
17 END WHILE
18
19 report ← CALL End-Session(session)
20 CALL Apply-SRS-Outcomes(report.routing)
21 RETURN report
END
```

The three server procedures called here — `Start-Session`, `Submit-Turn`, `End-Session` — are the feature's entire public surface. Everything else is in service of them.

Step 13 before step 15 is deliberate and is the single most important ordering in the whole feature: **the text is displayed before any audio exists**. Synthesis is slower than real time on the target hardware, so making the learner wait for sound before showing words cost roughly 30 seconds per reply.

Algorithm 1 LAUNCH-ENGINES

explicit, before any conversation

```
ALGORITHM Launch-Engines(learner)
INPUT   learner
OUTPUT  engine status; or FAILURE with a remediation message
EFFECT  loads models into memory, or verifies an API key

BEGIN
1  target ← CALL Resolve-LLM-Target(learner)
2
3  IF target.provider = LOCAL THEN
4      IF NOT model-file-exists(target.option) THEN
5          RAISE EngineUnavailable("no model downloaded - download one in Settings")
6      END IF
7      warm-up(target.option)           // load weights into RAM, tens of seconds
8  ELSE
9      IF target.apiKey = NIL THEN
```

```

10      RAISE EngineUnavailable("no API key configured - add one in Settings")
11    END IF
12    verify(target.apiKey, target.model)      // spend ONE small request
13    record target as verified                // so status polls never spend quota
14  END IF
15
16  IF stt-model-downloaded() THEN warm-up(STT) END IF
17
18  ttsName ← learner's selected TTS engine
19  IF tts-downloaded(ttsName) THEN
20    warm-up(ttsName)                        // ONLY the selected engine
21  END IF
22
23  RETURN engine-status(learner)
END

```

Two distinct ideas of "ready" appear throughout the feature and must not be confused:

Check	Question	Cost of failing to check
DOWNLOADED	Is the file on disk / is a key configured?	A multi-minute implicit download starts mid-conversation.
LOADED	Are the weights in memory?	A 30-second stall on the learner's first sentence.

Step 12 is the cloud equivalent of loading a model: one real request establishes that the key works. Step 13 records the result, because the status endpoint is polled and probing on every poll would spend the learner's quota to draw a status light.

Step 20's restriction to the selected engine avoids holding several hundred megabytes for a voice that will never be called.

Algorithm 2 START-SESSION

server

```

ALGORITHM Start-Session(learner, scenario, channel, seedWordIds)
INPUT  scenario in {free, coffee, job, debate}; channel in {voice, text}
       seedWordIds, optional - reuse the words from a session being repeated
OUTPUT sessionId
PRECONDITION  engines downloaded AND loaded
POSTCONDITION  either a session exists WITH turn 0, or nothing was written

BEGIN
1  ASSERT-DOWNLOADED(learner, channel)
2  ASSERT-LOADED({LLM} UNION (IF channel = VOICE THEN {TTS} ELSE {}))
3
4  IF seedWordIds != NIL THEN
5    words ← lookup(seedWordIds) scoped to learner    // drops deleted ids
6  END IF
7  IF words is empty THEN
8    words ← CALL Select-Target-Words(learner)
9  END IF
10
11  prompt ← CALL Build-System-Prompt(scenario, words, opening := NIL)
12  seed   ← [(USER, "(The learner has just joined. Greet them.)")]
13
14  // GENERATE BEFORE WRITING. See note.
15  greeting ← generate-reply(target, prompt, seed)
16
17  sessionId ← new uuid7()
18  INSERT session(sessionId, learner, scenario, channel,

```

```

19         targetWordIds := serialise(words), modelId := id-of(target))
20     COMMIT
21     INSERT turn(sessionId, index := 0, speaker := AI, text := greeting)
22     COMMIT
23     RETURN sessionId
END

```

Why step 15 precedes step 18 — two independent reasons.

- (a) If the session row were written first and generation then failed, the learner's history would hold a conversation with no turns in it, forever. "Resume" could only ever reopen it empty.
- (b) SQLite permits one writer at a time. Holding a write transaction open across an AI call — minutes, on a cold model load — blocks every other writer in the application for that whole period.

Step 12's seed message is synthetic. It is never stored and the learner never sees it; it exists only because the message list must begin with a user turn (Algorithm 4).

Step 21 means **every conversation begins with an assistant turn**, which is precisely the condition Algorithm 4 exists to repair.

Algorithm 3 SELECT-TARGET-WORDS

server

```

ALGORITHM Select-Target-Words(learner)
OUTPUT  at most 8 vocabulary words, least-practised first

BEGIN
1  FOR EACH word w IN vocabulary(learner) DO
2      usage[w] ← COUNT of review_logs WHERE word = w
3              AND outcome != 'avoided'           // <- the load-bearing filter
4  END FOR
5
6  sort vocabulary BY usage ASC, THEN BY createdAt DESC
7  RETURN first 8
END

```

Why step 3 excludes 'avoided'. An *avoided* log means the word was offered as a target and the learner never said it. If that counted as usage, the word would look practised and sink down the queue — so the words that most need work would be pushed to the back *precisely because* the learner keeps failing to use them. The filter makes only genuine production count.

The tie-break on `createdAt DESC` favours recently saved words, on the assumption that a word just looked up is one the learner is currently interested in.

Algorithm 4 NORMALISE-FOR-CHAT

server - called before every generation

```

ALGORITHM Normalise-For-Chat(pairs)
INPUT  pairs, a transcript as [(role, text)] where role in {user, assistant}
OUTPUT (opening, history) such that
    - history begins with a USER message
    - history strictly alternates user/assistant
    - opening holds any assistant text lifted off the front

BEGIN
1  merged ← empty list
2  FOR EACH (role, text) IN pairs DO
3      clean ← trim(text)

```

```

4      IF clean is empty THEN CONTINUE END IF
5      IF merged is non-empty AND last-role(merged) = role THEN
6          append clean to the last message      // MERGE, do not drop
7      ELSE
8          append (role, clean) to merged
9      END IF
10     END FOR
11
12     opening ← empty list
13     WHILE merged is non-empty AND first-role(merged) = ASSISTANT DO
14         move first message from merged into opening
15     END WHILE
16
17     RETURN (opening, merged)
END

```

This repairs two failures that were observed in production, not anticipated.

Failure 1 — the session opens with the AI speaking. Gemma's chat template requires strict user/assistant/user/... alternation beginning with *user*. Measured against gemma-3-1b: system + assistant + user raises "*Conversation roles must alternate*", while system + user + assistant + user succeeds. Since every conversation opens with a greeting (Algorithm 2, step 21), **every reply on every Gemma model failed**. Lines 13–15 lift those greetings out; the caller folds them into the system prompt instead, so their content survives without breaking the rule.

Failure 2 — two consecutive turns from one speaker. A failed generation used to leave a user turn with no reply. The next attempt then saw two user turns in a row and hit the same template error — which *masked* the original failure. Lines 5–6 merge same-role runs rather than dropping them.

Merging rather than dropping matters because orphaned turns are no longer *created* (Algorithm 5 was restructured), but sessions already in that state still exist in real databases. Fixing a cause does not remove the duty to keep old data working.

4. The turn loop

One exchange, across both processes: capture on the client, generation on the server, playback back on the client.

Algorithm 5 SUBMIT-TURN

server - the core of the feature

```
ALGORITHM Submit-Turn(session, text, audio)
INPUT    exactly one of text or audio
OUTPUT   (userTurn, aiTurn), both newly persisted
POSTCONDITION either BOTH turns were written, or NEITHER was

BEGIN
1  target ← CALL Resolve-LLM-Target(session.learner)  // current Settings, not pinned
2  ASSERT-DOWNLOADED(target, session.channel)
3
4  IF session.modelId ≠ id-of(target) THEN
5      UPDATE session.modelId ← id-of(target)          // record what ANSWERS, not what MUST
6  END IF
7
8  needs ← {LLM}
9  IF audio ≠ NIL THEN needs ← needs UNION {STT} END IF
10 IF session.channel = VOICE THEN needs ← needs UNION {TTS} END IF
11 ASSERT-LOADED(needs)
12
13 // ---- 1. Speech to text -----
14 IF audio ≠ NIL THEN
15     (text, confidence, speechSeconds) ← transcribe(audio)
16 END IF
17 text ← trim(text)
18 IF text is empty THEN
19     RAISE ValueError("Didn't catch any speech there - try again, louder")
20 END IF
21
22 // ---- 2. Rebuild the conversation for the model -----
23 turns ← all turns of session ORDER BY index
24 nextIndex ← (index of last turn) + 1, or 0 if none
25
26 pairs ← [ (IF t.speaker = AI THEN ASSISTANT ELSE USER, t.text) FOR t IN turns ]
27 append (USER, text) to pairs
28 (opening, history) ← CALL Normalise-For-Chat(pairs)
29 prompt ← CALL Build-System-Prompt(session.scenario, targetWords, opening)
30
31 // ---- 3. Generate BEFORE persisting -----
32 reply ← generate-reply(target, prompt, history, maxTokens := 120)
33
34 // ---- 4. Persist both turns together -----
35 userTurn ← INSERT turn(nextIndex, USER, text, confidence, speechSeconds)
36 aiTurn ← INSERT turn(nextIndex + 1, AI, reply)
37
38 RETURN (userTurn, aiTurn)  // NO audio synthesised here - see Algorithm 8
END
```

The postcondition is the whole design. If step 32 raises, nothing has been written and the database is exactly as it was. The learner sees an error and simply tries again.

Previously the user turn was written *first*. A failure then left it orphaned, and the next attempt saw two user

turns in a row — which Gemma rejects — so the session was **permanently stuck** behind an alternation error that said nothing about the original failure.

Step 19's guard. Speech-to-text returning empty means silence, too short a clip, or unclear speech. Sending an empty turn is meaningless to every provider, and Gemini rejects it with a confusing error about the conversation "ending with a model turn" — which is this problem in disguise, since the empty user part is dropped during validation, leaving the prior AI turn looking last. Failing honestly here produces a message the learner can act on.

Step 26's translation. The database stores `ai`; every LLM API expects `assistant`. The two vocabularies meet at exactly this line.

Step 1 is not pinned to the session. Sessions used to be locked to the engine they started on. When that engine became unusable — quota spent, key revoked, model deleted — the conversation could never continue and *no Settings change could rescue it*. Following current Settings trades cosmetic voice consistency for the ability to recover.

Algorithm 6 CAPTURE-UTTERANCE

client - hands-free voice activity detection

ALGORITHM Capture-Utterance()

EFFECT emits one audio blob per utterance; no button is pressed

CONSTANTS FRAME = 50ms SPEECH_FRAMES = 2 SILENCE = 400ms
MIN_UTTERANCE = 400ms MAX_UTTERANCE = 30s
IDLE_RESTART = 1500ms FLOOR_FRAMES = 10
ABS_MIN_THRESHOLD = 0.012 NOISE_MULTIPLE = 3

BEGIN

```
1  open microphone; start recording CONTINUOUSLY // so onset is never clipped
2  speaking ← FALSE ; loudFrames ← 0 ; noiseFloor ← 0
3
4  EVERY FRAME DO
5      rms ← sqrt(mean(sample^2)) over the frame
6
7      IF fewer than FLOOR_FRAMES seen THEN
8          noiseFloor ← running mean of rms // calibrate to THIS room
9          CONTINUE
10     END IF
11
12     threshold ← max(ABS_MIN_THRESHOLD, noiseFloor * NOISE_MULTIPLE) * scale
13
14     IF a turn is already in flight THEN
15         discard buffered audio ; speaking ← FALSE ; CONTINUE
16     END IF
17
18     IF rms > threshold THEN
19         loudFrames ← loudFrames + 1 ; lastLoudAt ← now
20         IF NOT speaking AND loudFrames ≥ SPEECH_FRAMES THEN
21             speaking ← TRUE ; speechStartedAt ← now
22             SIGNAL onSpeechStart() // the barge-in signal
23         END IF
24     ELSE
25         loudFrames ← 0
26         IF speaking AND now - lastLoudAt ≥ SILENCE THEN
27             EMIT recorded blob ; speaking ← FALSE ; restart recorder
28         ELSE IF NOT speaking AND now - recorderStart > IDLE_RESTART THEN
29             restart recorder // recycle idle pre-roll
30         END IF
31     END IF
```

```

32
33     IF speaking AND now - speechStartedAt > MAX_UTTERANCE THEN EMIT blob END IF
34 END EVERY
END

```

Three constants each encode a specific failure they prevent:

Constant	Prevents
SPEECH_FRAMES = 2	One loud frame is a keyboard clack or a chair creak. Two consecutive is speech.
SILENCE = 400ms	Shorter cuts the learner off mid-breath; longer is dead air they feel.
ABS_MIN_THRESHOLD	In a very quiet room the noise floor collapses toward zero and <i>everything</i> reads as speech.

Step 1 records continuously, and step 29 throws most of it away. Recording only after detecting speech would clip the first syllable — by the time two frames confirm speech, 100ms is already gone. So recording never stops and each blob carries pre-roll from before speech began. But that buffer must be *recycled*: during a barge-in, a long buffer would ship seconds of the AI's own reply to the transcriber as though the learner had said it.

The `scale` in step 12 is set to 2.4 while the AI is speaking and 1 otherwise. Raising the bar during playback is what stops the reply, leaking through the speakers, from reading as the learner interrupting — a feedback loop in which the AI interrupts itself.

Algorithm 7 PLAY-REPLY

client - chunked playback with barge-in

ALGORITHM Play-Reply(aiTurn)

INPUT aiTurn with chunkTexts, at most 2 entries

EFFECT plays each chunk in order; highlights words in time; cancellable

BEGIN

```

1  runId ← ++globalPlaybackId           // any newer run supersedes this one
2  stop any audio already playing
3  micThresholdScale ← 2.4               // deafen VAD to our own output
4  created ← empty list                  // every object URL, for release
5
6  pending ← FETCH-CHUNK(aiTurn, 0)     // synthesis starts server-side NOW
7  FOR i ← 0 TO total - 1 DO
8      url ← AWAIT pending
9      IF runId != globalPlaybackId THEN RETURN END IF // barge-in or newer reply
10     IF url = NIL THEN BREAK END IF
11
12     pending ← FETCH-CHUNK(aiTurn, i + 1) // PREFETCH next while this plays
13
14     words ← CALL Time-Words(chunkTexts[i])
15     play url to completion, advancing the highlight per words
16 END FOR
17
18 release every url in created           // else a few hundred KB leaks per turn
19 micThresholdScale ← 1
END

```

ALGORITHM Time-Words(text)

OUTPUT a start offset per word, summing to the chunk's duration

BEGIN

```

1  weights ← [ length(w) FOR w IN split(text) ]
2  RETURN cumulative offsets proportional to weights, scaled to duration
END

```

Step 12 is what removes the gap between sentences. Chunk $i+1$'s synthesis is requested before chunk i begins playing, so the server is working on the remainder while the opening sentence is audible.

Step 9 is the barge-in check, and it must be inside the loop. Cancelling only the clip currently playing would let the *next* chunk start moments later — the AI talking over a learner who had just interrupted it. Comparing against a monotonically increasing run id cancels the whole sequence, including a prefetch already in flight.

Time-Words is an estimate and is deliberately an honest one. Pocket TTS returns no word timings, so the offsets are interpolated from word length. Length-weighting is closer than an even split — longer words do take longer to say — but it is interpolation, not data, and it is not presented as more than that.

Algorithm 8 ENSURE-AUDIO-CHUNK

server - lazy synthesis with an atomic write

ALGORITHM Ensure-Audio-Chunk(turnId, text, index, engine)

OUTPUT a path to a COMPLETE wav file

INVARIANT a reader never observes a partially written file

BEGIN

```

1  chunks ← CALL Split-For-Streaming(text)
2  IF index < 0 OR index >= |chunks| THEN RAISE IndexError END IF
3
4  path ← audioDir / (turnId + "." + engine + "." + index + ".wav")
5  IF path exists THEN RETURN path END IF           // cached from a previous request
6
7  audio ← synthesise(chunks[index])                // slower than real time
8
9  tmp ← path + "." + new uuid7() + ".part"         // private to THIS call
10 TRY
11     write audio to tmp
12     atomic-rename(tmp, path)                     // os.replace
13 FINALLY
14     delete tmp if it still exists
15 END TRY
16 RETURN path

```

END

ALGORITHM Split-For-Streaming(text)

OUTPUT at most TWO pieces: the opening sentence, then the remainder

BEGIN

```

1  sentences ← split text on sentence boundaries
2  IF |sentences| <= 1 THEN RETURN sentences END IF
3  head ← sentences[0] ; rest ← join(sentences[1..])
4  IF length(head) < 12 THEN RETURN [head + " " + rest] END IF // too short to pay off
5  RETURN [head, rest]

```

END

The race that steps 9–12 prevent. The client fetches chunk $N+1$ while N is still audible, so two requests for the same chunk genuinely overlap — a replay, a retry, a second window. Writing straight to `path` lets a reader open the file *while another thread is still writing it* and serve a truncated WAV, which plays as a click or as silence. An atomic rename means a reader sees either no file or the whole file.

Why exactly two chunks, and not more. Measured on the reference machine (Ryzen 3 3200G, no AVX-512), synthesising only the first sentence rather than the whole reply took Pocket TTS from 2.05s to 1.09s. Further splitting buys nothing: each chunk carries a fixed per-call cost, and on Kokoro a four-way split measured *2.7s slower overall* while making the prosody audibly restart.

Step 4's engine in the filename. The audio *is* that engine's voice. Without the engine in the name, switching engines would keep serving the previous one's cached audio, so the change would appear to do nothing on every reply already on screen.

5. Analysis and scheduling

What happens when the learner leaves — and the step that makes any of it count as study.

Algorithm 9 END-SESSION

server

```
ALGORITHM End-Session(session)
OUTPUT  report
EFFECT  supersedes this session's review logs and reschedules every target word

BEGIN
1  ASSERT-LOADED({LLM})           // judgement needed; speech is not
2  turns      ← all turns of session
3  targetWords ← lookup(session.targetWordIds)
4  transcript  ← [ (t.speaker, t.text) FOR t IN turns ]
5
6  // ---- the JUDGED half -----
7  analysis ← CALL Generate-Analysis(transcript, targetWords)
8
9  // ---- the DETERMINISTIC half -----
10 band    ← learner's CEFR level, or "B1" if never placed
11 metrics ← CALL Compute-Metrics(turns, band)
12
13 // ---- reconcile the two -----
14 routing ← empty list
15 FOR EACH w IN targetWords DO
16     outcome ← CALL Verify-Word-Usage(w, analysis, turns)
17     append (w, outcome) to routing
18 END FOR
19
20 correct ← COUNT of routing WHERE outcome IN {spontaneous, prompted}
21 IF routing is empty THEN
22     accuracy ← NIL           // NOT zero. Nothing to be accurate about.
23 ELSE
24     accuracy ← round(100 * correct / |routing|)
25 END IF
26
27 report ← assemble(version := REPORT_VERSION, accuracy, metrics, analysis, routing)
28 UPDATE session SET reportJson ← report, endedAt ← now
29
30 // ---- feed the scheduler -----
31 DELETE review_logs WHERE session = session // SUPERSEDE, never append
32 CALL Apply-SRS-Outcomes(session.learner, routing)
33
34 RETURN report
END
```

Step 31 deletes before step 32 writes. Ending recomputes over the *whole* transcript, so a second end must **supersede** the first rather than add to it. Appending would make one conversation count twice — both in a word's usage history and in Algorithm 3's "not practised yet" ordering, where it would wrongly demote words the learner has practised only once.

A session that has been ended is not closed. The learner may keep talking and end again later; the report is a checkpoint, not a seal.

Step 22 once more. `NIL`, not zero — a session with no target words did not score zero accuracy; it had nothing to be accurate about. This distinction runs through every metric in the feature.

Algorithm 10 VERIFY-WORD-USAGE

server - checking the model against the transcript

```
ALGORITHM Verify-Word-Usage(word, analysis, turns)
OUTPUT outcome in {spontaneous, prompted, incorrect, avoided}
PRINCIPLE the model is overruled exactly where evidence contradicts it,
           and not one step further

BEGIN
1  userTurns ← turns WHERE speaker = USER
2  aiTurns   ← turns WHERE speaker = AI
3
4  // A FACT about the transcript, never asked of the model:
5  evidenceTurn ← index of the FIRST t IN userTurns
6               WHERE Says-Word(t.text, word), else NIL
7
8  outcome ← Match-Word-Usage(analysis.wordUsage, word), else 'avoided'
9
10 IF evidenceTurn != NIL AND outcome = 'avoided' THEN
11   // The model claims it was never used; the transcript shows otherwise.
12   saidFirstByAI ← EXISTS t IN aiTurns
13                 WHERE t.index < evidenceTurn AND Says-Word(t.text, word)
14   IF saidFirstByAI THEN outcome ← 'prompted'
15   ELSE               outcome ← 'spontaneous'
16   END IF
17 END IF
18
19 RETURN outcome

END

ALGORITHM Says-Word(text, word)
OUTPUT TRUE iff text contains word AS A WORD
BEGIN
1  wanted ← base-forms(word) // "goes", "going" -> "go"
2  RETURN EXISTS w IN words(text) WHERE wanted INTERSECT base-forms(w) != {}

END
```

The bug this corrects. gemma-3-1b marked every word avoided in a session where the learner had plainly used all three. The report read 0/3, the words were logged as unpractised, and Algorithm 3 pushed them back to the front of the queue they had just earned their way out of. The learner was penalised for the model's mistake.

Only `avoided` is corrected, and the restraint is deliberate. `incorrect` is a judgement about *how* a word was used, and nothing available here can second-guess it. `avoided` means "never used at all" — the only verdict that is *checkable* against the transcript.

`Says-Word` must not be a substring test. "cat" occurs inside "category" and "art" inside "start" — enough to credit a learner with a word they never said, and to point the report's evidence link at the wrong turn. Matching on shared base forms gives one consistent notion of "the same word" across the whole report.

Algorithm 11 COMPUTE-METRICS

server - the deterministic half, a pure function

```
ALGORITHM Compute-Metrics(turns, band)
OUTPUT TranscriptMetrics
```

PROPERTY pure - same input, same output; no database, no model, no clock

BEGIN

```
1  userTurns ← turns WHERE speaker = USER      // the AI is never measured
2  allWords  ← concat(words(t.text) FOR t IN userTurns)
3  total     ← |allWords|
4
5  fillers   ← COUNT of allWords IN {um, uh, erm, ah, hmm, mmm, like, basically, actually}
6  fillerRate ← IF total > 0 THEN round(100 * fillers / total, 1) ELSE 0
7
8  speechSecs ← SUM of t.speechSeconds FOR t IN userTurns      // REAL speech time
9  IF speechSecs > 0 AND total > 0 THEN
10     wpm ← round(total / (speechSecs / 60))
11 ELSE
12     wpm ← NIL      // a text session HAS no speech rate
13 END IF
14
15 delays ← empty list
16 FOR EACH consecutive pair (prev, cur) IN turns DO
17     IF prev.speaker = AI AND cur.speaker = USER THEN
18         append (cur.createdAt - prev.createdAt) to delays    // excludes AI thinking time
19     END IF
20 END FOR
21 avgDelay ← mean(delays), or NIL if empty
22
23 longestRun ← max(|words(t.text)| FOR t IN userTurns), or 0
24
25 lemmas ← { normalise(w) FOR w IN allWords }
26 ttr    ← IF total > 0 THEN |lemmas| / total ELSE 0
27
28 aboveLevel ← sorted({ w IN lemmas WHERE cefr-band(w) > band })
29
30 confidences ← [ t.sttConfidence FOR t IN userTurns WHERE it is present ]
31 pronunciation ← IF confidences non-empty THEN round(100 * mean(confidences)) ELSE NIL
32
33 RETURN all of the above
END
```

Step 8 is why the database carries a `speech_seconds` column at all. Dividing by wall-clock time would include the model thinking and the AI talking — none of which is the learner speaking. That is not a speech rate; it is a measurement of the hardware.

Step 18's window is narrow on purpose. Only AI → learner gaps count, so the AI's own generation time sits outside the measurement. Averaging *every* turn gap would have made a slow model look like a hesitant learner.

Step 26, type-token ratio, is distinct lemmas over total words: 50 words using 40 distinct ones scores 0.8; one word repeated 50 times scores 0.02. Reducing to lemmas first means "go" and "goes" count once, matching `Says-Word`.

Three of the eight outputs can be `NIL`: `wpm`, `avgDelay` and `pronunciation`. Each is genuinely unmeasurable in some legitimate session, and none is ever reported as zero instead.

Algorithm 12 GENERATE-ANALYSIS

server - the judged half

ALGORITHM Generate-Analysis(transcript, targetWords)

OUTPUT LlmReportAnalysis, possibly empty, never invalid

```

BEGIN
1  // The prompt is RENDERED FROM the schema that will parse the reply.
2  shape ← join over fields F of LlmReportAnalysis: ("F.name": F.description)
3  systemPrompt ← "return ONLY a JSON object with exactly these keys: {" + shape + "}"
4                + definitions of spontaneous | prompted | incorrect | avoided
5                + "judge only the learner's turns, never the AI's"
6  userPrompt ← rendered transcript
7
8  raw ← generate-json(target, systemPrompt, userPrompt, maxTokens := 700)
9
10 TRY
11     RETURN LlmReportAnalysis.validate(raw)
12 CATCH ValidationError
13     RETURN LlmReportAnalysis()           // all defaults; report still renders
14 END TRY
END

ALGORITHM Match-Word-Usage(wordUsage, word)
OUTPUT the model's verdict for word, matched forgivingly, else NIL
BEGIN
1  wanted ← base-forms(word)
2  FOR EACH (key, outcome) IN wordUsage DO
3      IF wanted INTERSECT base-forms(key) != {} AND outcome IS VALID THEN
4          RETURN outcome
5      END IF
6  END FOR
7  RETURN NIL
END

```

Step 2 is the mechanism that keeps the prompt honest. The prompt is generated *from the very class that parses the response*. Add a field and the prompt asks for it automatically; rename one and both sides change together.

These had drifted apart once: the prompt asked for three keys while the reader expected six, leaving **four report fields permanently zero on every real session while the tests passed** — the tests fed well-formed data straight to the parser and never exercised the prompt.

Steps 10–14 are tolerant about what is missing and strict about what is extra. A small model will omit a key or mistype a value, and a partial analysis is still worth showing — so an invalid response yields an empty analysis rather than a failed report. What it may *not* do is invent a field nobody asked for.

Match-Word-Usage is forgiving for a measured reason. Asked to echo words back as JSON keys, a small model does not reliably do so verbatim — it lowercases a capitalised word, or answers about "wonders" when asked about "Wonder". An exact lookup turns that into a silent miss reading as `avoided`, indistinguishable from the learner never trying.

Algorithm 13 APPLY-SRS-OUTCOMES

server - why any of this counts as study

```

ALGORITHM Apply-SRS-Outcomes(learner, routing)
EFFECT reschedules each target word through the SAME FSRS path a flashcard uses

BEGIN
1  FOR EACH (word, outcome) IN routing DO
2      rating ← CASE outcome OF
3          'spontaneous' -> EASY
4          'prompted'    -> GOOD
5          'incorrect'   -> AGAIN

```

```

6         'avoided'      -> HARD
7     END CASE
8
9     card ← FSRS card for (learner, word)
10    card ← fsrs-review(card, rating, now)    // identical call to a flashcard answer
11    SAVE card
12    INSERT review_log(word, outcome, source := 'conversation', session)
13 END FOR
END

```

This is the project's actual thesis, expressed in a **CASE statement**. A word produced out loud in conversation reschedules *exactly* as the same word recognised on a flashcard would, because it travels through the same FSRS function with the same kind of rating. Without step 10, the report would be a log nobody acts on.

Why **avoided** maps to **HARD** and not **AGAIN**. **AGAIN** pushes a card into relearning, which is too severe for a word the learner may simply never have had an opening to use. Avoiding a word is weak evidence of difficulty, not evidence of failure.

Step 12's source field is load-bearing. Once flashcard reviews began sharing this table, a query without it counted both — so the Vocabulary page reported *"conversation usage: again 1x"* for a word only ever answered on a flashcard. "Again" is not something anyone can do in a conversation.

6. Invariants and cost

Seven properties hold across the algorithms. Most were established by a failure rather than by design, and each names what it prevents.

#	Invariant	Held by	Prevents
I1	Generate before persist. No turn is written unless its reply exists.	2 step 15 5 step 32	An orphaned user turn, which makes the next request fail an alternation check that hides the real error.
I2	History alternates, starting with the user.	4	Outright rejection by Gemma and Gemini chat templates.
I3	NIL is never rendered as 0.	9 step 22 11 steps 12, 21, 31	Reporting a speaking rate for a session in which nobody spoke.
I4	Evidence outranks judgement, where evidence exists.	10 steps 10–17	A model's mistaken "avoided" demoting words the learner had actually used.
I5	The prompt is generated from the parser's schema.	12 step 2	Silent drift between keys asked for and keys read.
I6	Readers never see a partial file.	8 steps 9– 12	A truncated WAV served mid-write, heard as a click or silence.
I7	Re-ending supersedes; it never appends.	9 step 31	One conversation counted twice in a word's practice history.

Cost of one spoken turn

Where the learner's waiting actually goes, on the reference machine (Ryzen 3 3200G, 4 cores, no AVX-512):

Stage	Order	Note
VAD end-of-speech	400 ms	Fixed. Shorter cuts the learner off mid-breath.
Transcription	~ length of utterance	faster-whisper tiny.en, local.
Reply generation	dominant term	Capped at 120 tokens for this reason.
Synthesis, chunk 0	~1.1 s	2.05 s before the split. This is the only synthesis the learner waits on.
Synthesis, chunk 1	hidden	Overlapped with chunk 0's playback (7 step 12).

Two optimisations follow directly from this table, and neither is cosmetic: **text is displayed before any audio exists**, and **only the first sentence is synthesised before responding**. Together they turn a ~30 second wait into roughly one second plus generation.

The single idea underneath all fourteen algorithms: keep a sharp line between what can be *computed* and what must be *judged*, and never let the model's guesses leak into the half of the report that is supposed to be fact. The NIL discipline, the transcript checks that overrule the model, the self-generating prompt and the base-form matching all exist to defend that line.